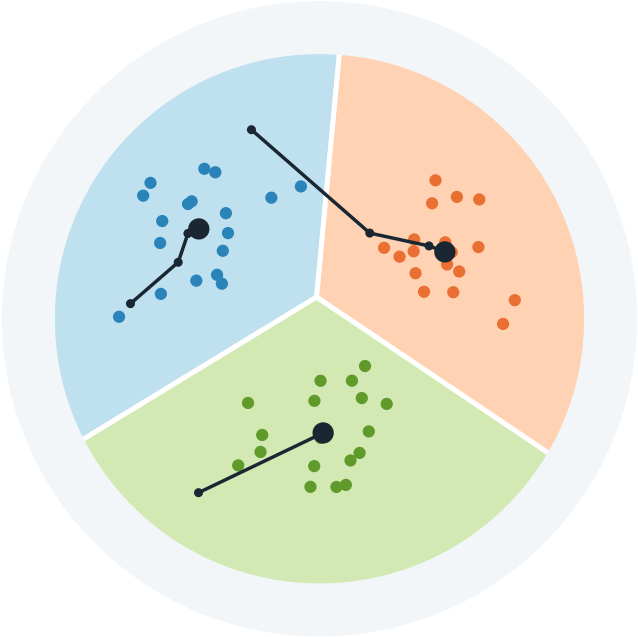
k-средних

Урок



Пятый, последний из уроков о моделях — модель без ответов. Из урока «Ближайшие соседи» здесь нужны расстояния и стандартизация признаков. Пример — те же двенадцать пингвинов в стандартных единицах; виды понадобятся только в конце, чтобы сравнить с ними найденные группы:

```
import numpy as np
# клюв, мм; масса, г; вид - 0, 1, 2
X = np.array([[38, 3600], [40, 3800], [36, 3300],
              [41, 3950], [47, 3550], [50, 3800],
              [46, 3400], [51, 3950], [46, 4900],
              [49, 5400], [41, 4650], [50, 5600]])
y = np.array([0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2])
Z = (X - X.mean(axis=0)) / X.std(axis=0)
```

1 k-средних

Бывает, что ответов нет и в объектах надо найти группы — *кластеры*. Метод *k-средних* ищет k центров $\mu_1, \ldots, \mu_k$ и номер кластера c_i у каждого объекта так, чтобы сумма квадратов расстояний от объектов до центров их кластеров была мала:

$$\text{SSE} = \sum_{i=1}^{n} \|x_i - \mu_{c_i}\|^2.$$

Разбиений около k^n, их не перебрать. *Алгоритм Ллойда* улучшает по очереди то разбиение, то центры: *назначение* — каждый объект в кластер ближайшего центра; *пересчёт* — каждый центр в среднее объектов своего кластера. Шаги повторяют, пока назначение не перестанет меняться.

Назначение — таблица квадратов расстояний «объект — центр» и номер наименьшего в строке. Пингвины в

стандартных единицах, стартовые центры — пингвины 0, 4 и 8; последняя строка — SSE этого назначения:

```
C = Z[[0, 4, 8]]
diff = Z[:, None, :] - C[None, :, :]
D = (diff ** 2).sum(axis=2)
c = D.argmin(axis=1)
print(D.shape, c)
print(D.min(axis=1).sum().round(3))
```

```
(12, 3) [0 0 0 0 1 1 1 1 2 2 2 2]
6.121
```

Если в кластер не попал ни один объект, он *пуст*, и среднее не определено: оставить его центр, убрать кластер или перенести центр в далёкий объект — отдельное правило алгоритма.

Пример (на прямой). Объекты 1, 2, 4, 7, 8, 12, $k = 2$, стартовые центры — 1 и 2. В строке — кластеры после назначения и SSE при старых центрах, затем центры после пересчёта и SSE при них:

шаг	назначение	SSE	центры	SSE
1	$\{1\}$, $\{2, 4, 7, 8, 12\}$	165	1; 6,6	59,2
2	$\{1, 2\}$, $\{4, 7, 8, 12\}$	39,04	1,5; 7,75	33,25
3	$\{1, 2, 4\}$, $\{7, 8, 12\}$	25,4375	$\frac{7}{3}$; 9	$18\frac{2}{3}$

Четвёртое назначение ничего не меняет: 7 ближе к 9, чем к $\frac{7}{3}$. Останов.

Утверждение. SSE не растёт ни при назначении, ни при пересчёте. Если при равных расстояниях объект остаётся в своём кластере, алгоритм останавливается за конечное число шагов.

Почему так. При назначении центры неподвижны, каждый объект выбирает ближайший, и его слагаемое не растёт. При пересчёте неподвижно разбиение, слагаемые кластера S —

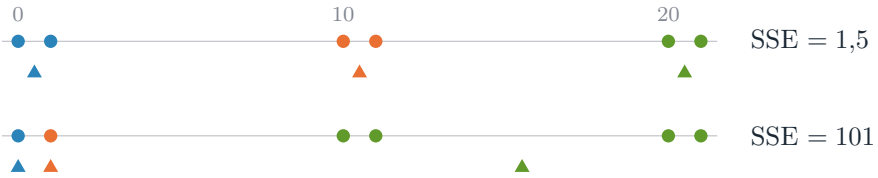
это $\sum_{i \in S} \|x_i - \mu\|^2$, и среднее $\bar{x}$ даёт наименьшую такую сумму: для любого μ

$$\sum_{i \in S} \|x_i - \mu\|^2 = \sum_{i \in S} \|x_i - \bar{x}\|^2 + |S| \cdot \|\bar{x} - \mu\|^2,$$

потому что $\|x_i - \mu\|^2 = \|x_i - \bar{x}\|^2 + 2(x_i - \bar{x}) \cdot (\bar{x} - \mu) + \|\bar{x} - \mu\|^2$, а $\sum_{i \in S}(x_i - \bar{x}) = 0$. Останов: если назначение изменилось, какой-то объект перешёл к строго более близкому центру, и SSE строго уменьшилась. Значит, разбиения не повторяются, а их конечное число.

Локальные минимумы и старты

Но приходит алгоритм в *локальный минимум* — состояние, которое шаги не меняют, хотя SSE не наименьшая. Объекты 0, 1, 10, 11, 20, 21, $k = 3$, старт в объектах 0, 1 и 11: центры приходят в 0, 1 и 15,5 и стоят — объект 10 ближе к 15,5, чем к 1. SSE — 101, а у разбиения на пары — 1,5:



Поэтому k-средних запускают много раз с разных стартов и берут результат с наименьшей SSE. Старт случайный: k случайных объектов или *k-means++* — первый центр случайный объект, каждый следующий — объект, взятый с вероятностью, пропорциональной квадрату расстояния до ближайшего уже выбранного центра. Так центры разносятся по данным, и старты вроде показанного редки. Расстояния здесь те же, что у соседей в уроке «Ближайшие соседи», и масштаб важен так же: признаки стандартизуют.

Как сравнить два разбиения

Номера кластеров произвольны: разбиения $(0,0,1,1)$ и $(1,1,0,0)$ — одно и то же. Поэтому сравнивают пары объектов: разбиения согласны на паре, если в обоих она вместе или в обоих врозь. Доля согласных пар — *индекс Рэнда*; беда в том, что и случайные разбиения согласны на многих парах, больше всего — на парах «врозь».

Скорректированный индекс Рэнда, ARI, вычитает то, что дала бы случайность. Пусть n_{ij} — сколько объектов в кластере i первого разбиения и в кластере j второго, a_i и b_j — суммы этой таблицы по строкам и столбцам, $\binom{m}{2} = m(m-1)/2$ — число пар из m. Тогда

$$\mathrm{ARI} = \frac{\sum_{ij}\binom{n_{ij}}{2} - E}{\frac{1}{2}\left(\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right) - E}, \qquad E = \frac{\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}}{\binom{n}{2}}.$$

В числителе — сколько пар вместе в обоих разбиениях, минус сколько их вышло бы в среднем, если перемешать объекты при тех же размерах кластеров; в знаменателе — то же для верхней границы этого числа. $\mathrm{ARI} = 1$, когда разбиения совпадают с точностью до номеров, около 0 — когда не связаны; бывает и отрицательным.

Пример (шесть объектов). Разбиения $(0,0,0,1,1,1)$ и $(0,0,1,1,2,2)$. Строки таблицы n_{ij} — $(2,1,0)$ и $(0,1,2)$, вместе в обоих — $1+1=2$ пары. $a=(3,3)$, $b=(2,2,2)$, $\sum\binom{a_i}{2}=6$, $\sum\binom{b_j}{2}=3$, $E = 6\cdot 3/15 = \frac{6}{5}$, и

$$\mathrm{ARI} = \frac{2-\frac{6}{5}}{\frac{1}{2}(6+3)-\frac{6}{5}} = \frac{8}{33} \approx 0{,}24.$$

Индекс Рэнда — $\frac{10}{15}$: две пары вместе в обоих и восемь врозь в обоих.

2 То же в sklearn

Порядок вызовов — как в уроке «Ближайшие соседи»: создать объект с гиперпараметрами, `fit(X)` — здесь без ответов. Номера кластеров — в `labels_`, центры — в `cluster_centers_`.

```
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score

for data in [X, Z]:
    km = KMeans(n_clusters=3, n_init=10,
                random_state=0).fit(data)
    ari = adjusted_rand_score(y, km.labels_)
    print(km.labels_, round(ari, 3))
print(round(km.inertia_, 3))
```

```
[0 0 0 0 0 0 0 0 2 1 2 1] 0.368
[0 0 0 0 1 1 1 1 2 2 0 2] 0.737
4.417
```

В граммах кластеры — полосы по массе. В стандартных единицах — почти виды: папуанский $(41, 4650)$ с коротким клювом ушёл к Адели, и SSE этого разбиения, `inertia_`, меньше, чем у разбиения по видам, 5,079: k-средних ищет компактные группы, а не виды. Свой старт — массив центров в `init` при `n_init=1`; на прямой из раздела 1:

```
line = np.array([[0], [1], [10], [11], [20], [21]])
start = np.array([[0], [1], [11]])
for init in [start, "k-means++"]:
    km = KMeans(3, init=init, n_init=1,
                random_state=0).fit(line)
    print(km.cluster_centers_.ravel(), km.inertia_)
```

```
[ 0.   1.  15.5] 101.0
[10.5  0.5 20.5] 1.5
```

Ловушка. `KMeans` не точный Ллойд. `n_init` по умолчанию `"auto"` — для k-means++ это *один* запуск со случайным стартом, его фиксирует `random_state`. Останов — когда назначение не изменилось *или* центры сдвинулись мало: сумма квадратов сдвигов не больше `tol` (10^{-4}) на среднюю дисперсию признаков; шагов не больше `max_iter=300`. После останова по `tol` объекты назначаются ещё раз, а центры — нет; точно «до неизменного назначения» — `tol=0`. Центр пустого кластера sklearn переносит в объект, самый далёкий от своего центра.

Итог пяти уроков. Соседи хранят таблицу, дерево учит вопросы, линейные модели — веса, k-средних — центры; масштаб не важен только дереву. Качество меряют на отложенных данных, гиперпараметры выбирают скользящим контролем, правило ничьих — часть модели. Дальше — метрики по файлу предсказаний и ансамбли: лес и бустинг строятся из деревьев.